

AI Engineer Take-Home Project

Utility Invoice Processing with LLMs

Note: Make any assumptions necessary to complete the project — and explain your assumptions.

Objective

Build a lightweight system that extracts structured data from utility invoices (electricity, gas, or water) using an LLM, and outputs the results as a CSV file. This exercise is designed to evaluate your ability to:

- Work with LLMs in a practical data extraction task
- Design simple but effective pipelines
- Handle multilingual inputs
- Make pragmatic engineering tradeoffs under time constraints

Project Requirements

Input Data

A small set of sample utility invoices (PDFs). Include at least 3–5 real-world utility bills/invoices with variation in:

- Format/layout
- Language (at least 2 different languages, e.g., English + Spanish/French/etc.)

You may use publicly available examples or create mock invoices.

UI Application

Create a web app with UI that allows uploading PDF documents for the invoices from which the invoice data is to be extracted and saved in a CSV file.

Output format

Generate a CSV file with the following fields (at minimum):

- **vendor_name:** Name of the utility provider
- **invoice_date:** Date the invoice was issued
- **service_address:** Location where services were rendered (if available)
- **utility_type:** Type of utility (electricity, gas, water)
- **usage_amount:** Consumed quantity (numeric)
- **usage_unit:** Unit of measurement (e.g., kWh, therms, gallons)

- **billing_period_start:** Start date of the billing cycle
- **billing_period_end:** End date of the billing cycle

Core Task & Pipeline Workflow

Build a pipeline that achieves the following steps:

- Ingests invoice documents through the UI application workflow.
- Extracts text directly from the documents (assume text is already available or can be extracted; OCR is not strictly required).
- Uses an LLM to parse and structure the data.
- Handles multilingual content appropriately.
- Outputs a clean CSV file and allows the user to download or save it.

You may assume and design around:

- Variation in invoice layouts
- Multiple languages
- Missing or ambiguous fields

Tools & Technology Stack

You are encouraged to use modern "vibe coding" tools and AI-assisted environments such as:

- Claude Code / Replit / Cursor / VS Code + Copilot
- Gemini / GPT / Claude LLM

You may use any libraries/frameworks you prefer (e.g., Python, Node.js, LangGraph, Streamlit, Gradio, Next.js, etc.) to fulfill both the backend pipeline and the frontend UI.

What We're Looking For

We are not expecting production-grade enterprise systems. Instead, we evaluate based on:

- Clarity of approach
- Full stack design and development
- Thoughtful use of LLMs (prompting, multilingual handling, structure, reliability)
- Handling of variability across formats and languages
- Simplicity and pragmatism
- Code readability

Bonus Objectives (Optional)

- Language detection or normalization strategy
- Confidence scoring for extracted elements

- Basic validation checks (e.g., date formats, numeric validation)
- Modular or extensible pipeline design

Deliverables & Submission Guidelines

Please submit the following items:

- 1. Code Repository:** Well-organized code containing both the processing pipeline and the UI app.
 - **Instructions:** Clear setup instructions to run the web application locally.
 - **Sample Files:** Include the 3–5 sample input files used for validation.
- 2. Output CSV:** Generated output from your sample invoices.
- 3. Testing Approach:** A short section (in your README or as a separate document) describing how you validated accuracy, edge cases considered (missing fields, language/unit variations), automated tests implemented (optional), and future testing improvements.
- 4. Short Video Walkthrough:** A 5–10 minute walkthrough video covering your approach, key design decisions, UI app demo, system architecture, multilingual handling, tradeoffs made, future improvements, and key project takeaways.

Evaluation Criteria

- Problem-solving approach
- Full stack development process
- Effective use of LLMs and robust prompting
- Handling of multilingual data
- Code quality, organization, and modularity
- Testing and validation thinking
- Communication clarity (especially in the video walkthrough)